Spurs² Content Intelligence

Spurs² Content Intelligence — Solution Overview & Reviewer Guide

Challenge addressed: *Assisted Auto-Production Engine* — "one game, a thousand outputs." **Built by:** Clout Labs
Stack: ScorePlay (media system of record) · Twelve Labs (Marengo + Pegasus video understanding) · Anthropic Claude (reasoning & copy) · Postgres/Supabase · Next.js

> **The one-line thesis:** the producer **approves** instead of **searches.** The system watches the match, decides what's worth making, cuts it for every audience and channel, shows its reasoning (including what it *rejected* and why), and puts the right five options in front of a producer in the time it used to take to find one.

---

1. The problem we chose

Clubs capture enormous amounts of footage and can use only a fraction of it. What gets made is limited by how many editors are free — while fans have already decided what matters, often before anyone in the building has found the clip. The brief asked for *"the system that decides what is worth making and then makes it, with a producer approving rather than searching,"* and — critically — *"show your work, including what you left out."*

That last requirement shaped the whole product.

2. What the system does

1. **Watches the match.** Every scoring/notable moment is detected and cut automatically — from the broadcast, and extensible to ISO cameras, stills and archive.
2. **Scores each moment** across *significance*, *audience fit*, and *corroboration* (including signals from outside the video — crowd reaction, bench, the room going quiet).
3. **Produces, not just finds.** One moment becomes many outputs: reels, photo cards, and per-channel cuts — each framed to the right aspect ratio with safe zones, for up to **9 channels**.
4. **Explains itself ("Why this clip").** For every proposal the producer sees the play it saw, the signals behind the score, the audience it's for — and the system also surfaces **what it considered and rejected, with a reason**, so a human can catch the gem the model missed.
5. **Keeps a human in the loop.** The producer approves, edits, or rejects. Rights are checked against ScorePlay (e.g. `can_be_shared`, owned-channels-only) and re-checked again at publish.
6. **Personalizes for fans.** Approved moments flow into a fan app whose feed re-ranks per supporter (e.g. "follows Solanke & goals" vs "crowd & atmosphere") and supports natural-language search over the footage.

3. How to explore it (three ways, easiest first)

A. Watch the 73-second walkthrough — *recommended*

`submission/Spurs2-Content-Intelligence-Demo.mp4` — a narrated tour of the **actual running system** (not slides): producer inbox → review a clip with video playback → "Why this clip" reasoning → one-moment-to-nine-channels → approve → **a live Twelve Labs semantic search** → the personalized fan app.

B. Run it locally

Prerequisites: Node 20+, Docker, and the API keys in `.env` (Twelve Labs + ScorePlay + Anthropic).

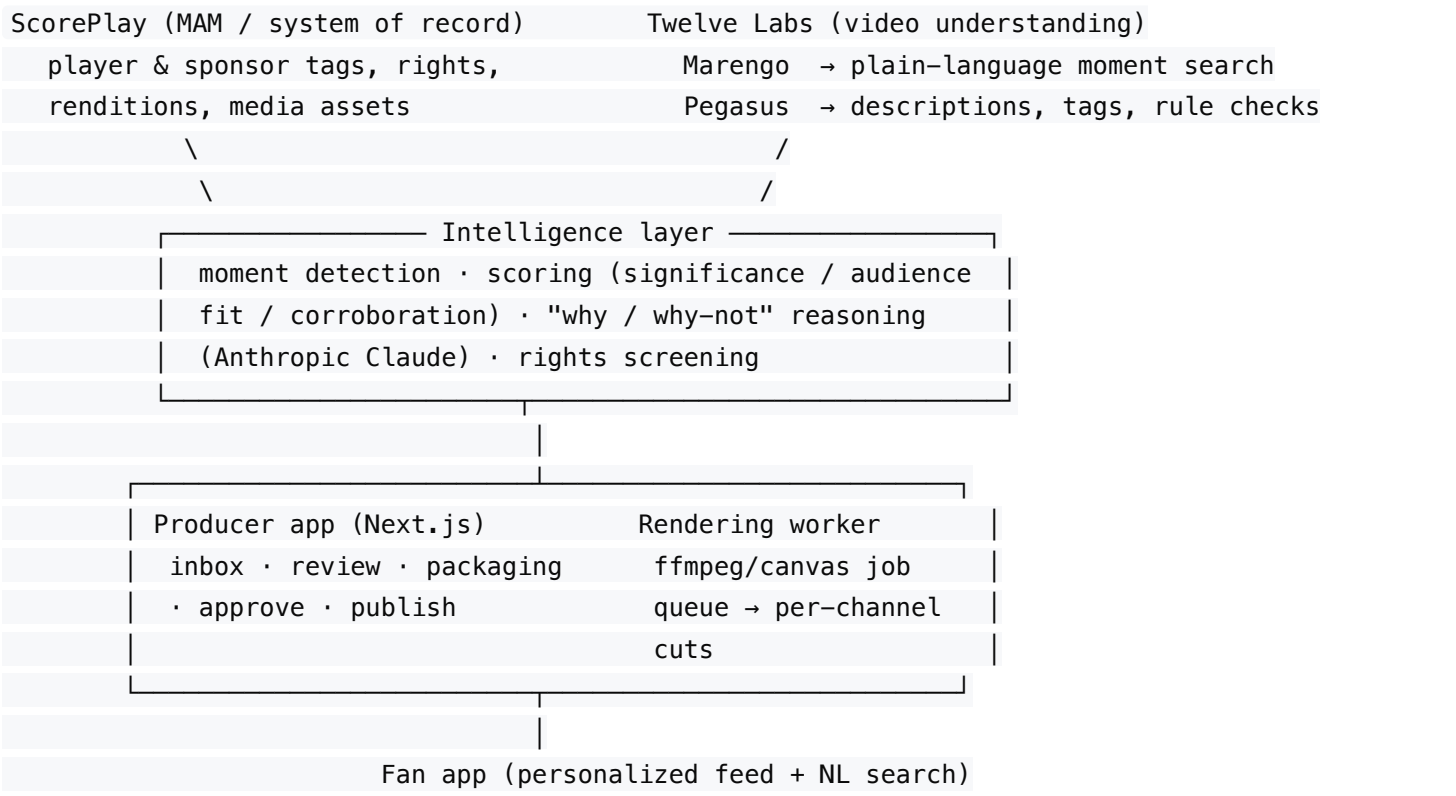
```
# one command: starts a local Postgres, seeds the demo state, builds and serves the app + worker
NO_OPEN=1 bash scripts/demo.sh
# then open:
open http://localhost:3000        # producer app
open http://localhost:3000/fan    # fan app
open http://localhost:3000/library  # live Twelve Labs search
```

Guided tour once it's up: - **Inbox (`/`)** — the scored queue. Pick any clip. - **Review pane** — press play; open **Why this clip**, then **Channels** to see the 9-channel packaging, then **Fan app**. Click **Approve**. - **Library (`/library`)** — click a search chip (e.g. "Wembanyama dunk"); results return **live** from Twelve Labs against the real match index. - **Fan app (`/fan`)** — toggle the fan personas and watch the feed re-rank.

C. (Optional) Hosted link

If the submission requires a URL, a hosted, browse-only build can be provided — see §6 for exactly what is and isn't possible to host, so there are no surprises.

4. Architecture



```
  State: Postgres / Supabase (22 tables)   Every Twelve Labs asset carries its
                                           ScorePlay asset ID for write-back.
```

5. What's real vs. simulated (truth rule)

We label integrations honestly.

Capability	Status
Twelve Labs Marengo semantic search	**Live** — real queries, real results against the real indexes
Twelve Labs Pegasus descriptions/tags	**Live** integration
Scoring, "why/why-not" reasoning, copy (Claude)	**Live** integration
Producer workflow, packaging, fan personalization, rights screening	**Fully built and working**
ScorePlay auth, tag/player resolution, rights flags	**Live**
ScorePlay **media assets / streaming**	**Cleared post-event** — the hackathon platforms were wiped, so match playback in the demo uses clips rendered during the event (cached locally). The ScorePlay *read*/*write* integration itself is intact.

6. Hosting notes (honest constraints)

The app is production-shaped, not a static site. Three things determine what a hosted version can do: 1. **Video serving** reads the local filesystem (~1.2 GB of rendered clips). Hosting requires moving clips to object storage (Supabase Storage / Vercel Blob) and repointing playback. 2. **The rendering worker** is a long-running ffmpeg/canvas process — it needs a container (e.g. AWS Fargate), not serverless. A hosted web-only deploy is therefore **browse-only**: it shows already-produced content but doesn't render new clips live. 3. **Live-match streaming** depended on ScorePlay media that's now cleared, so that page is degraded in any environment until fresh event keys/footage are issued.

None of this limits the *local* experience, which is complete.

7. Production path

- Move rendered media to object storage + CDN; run the worker on a container service.
- Wire ScorePlay webhooks so a new asset lands → Twelve Labs → scored → queued automatically.
- Write verdicts/decisions back to ScorePlay as timecoded actions (closing the MAM loop).
- Per-tenant isolation so each club runs its own instance on its own brand.